

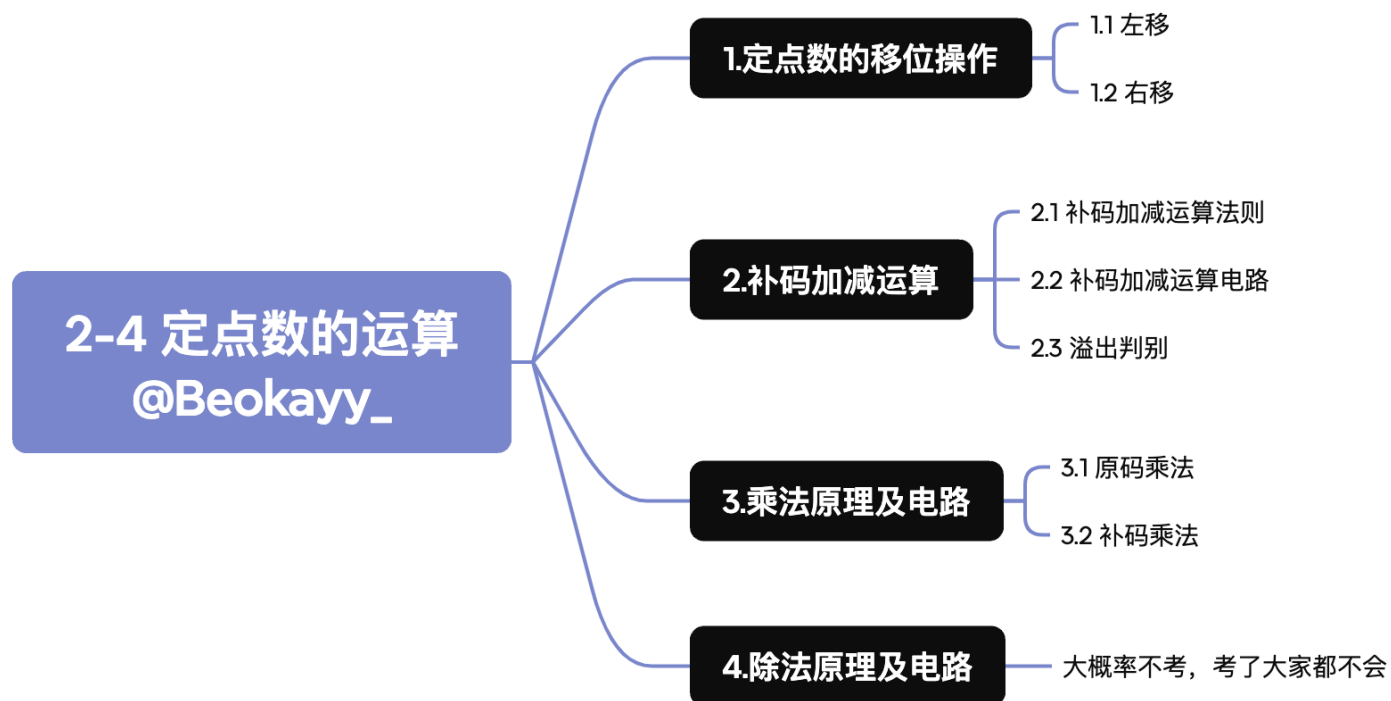
bok25

**基
础**

班、计算机组成原理



定点数的运算



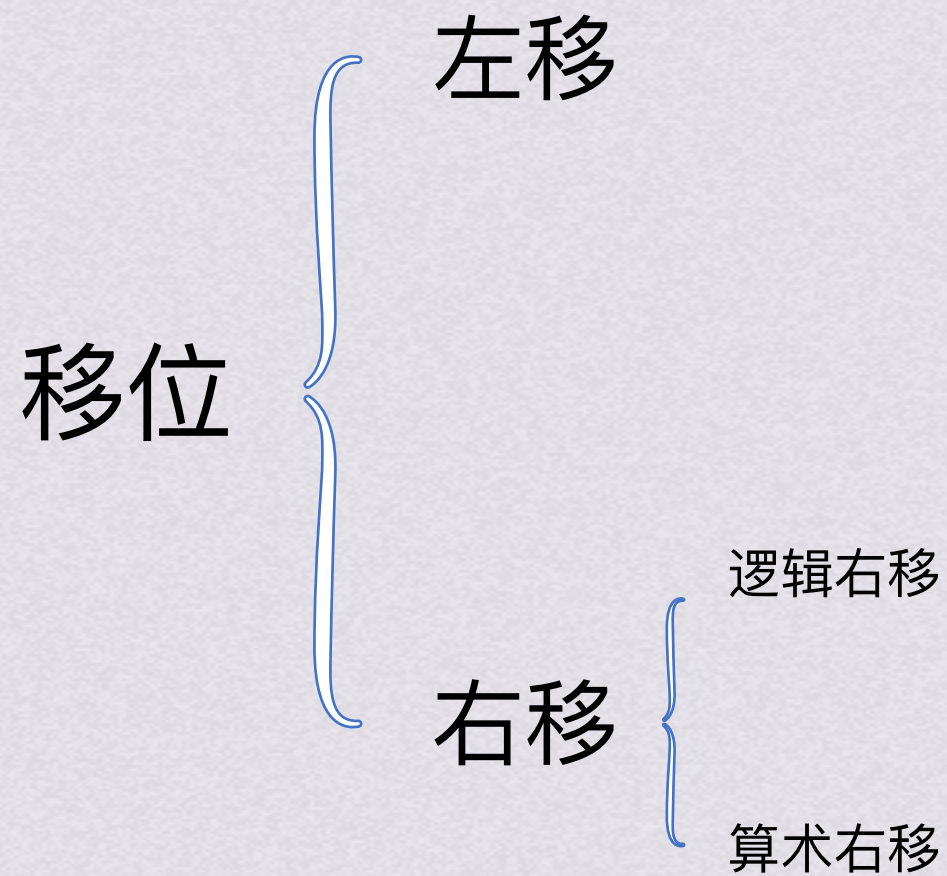


1.定点数的移位操作



定点数的运算

1. 定点数的移位操作





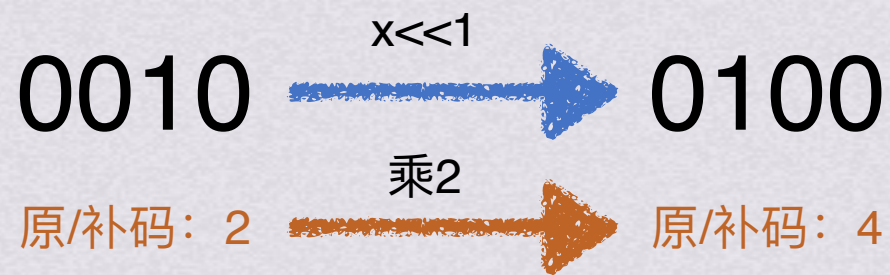
定点数的运算

1.定点数的移位操作 1.1 左移

左移

$$x \ll k$$

x向左移动k位，丢弃最高的k位，并在右端补k个0



左移相当于乘2



定点数的运算

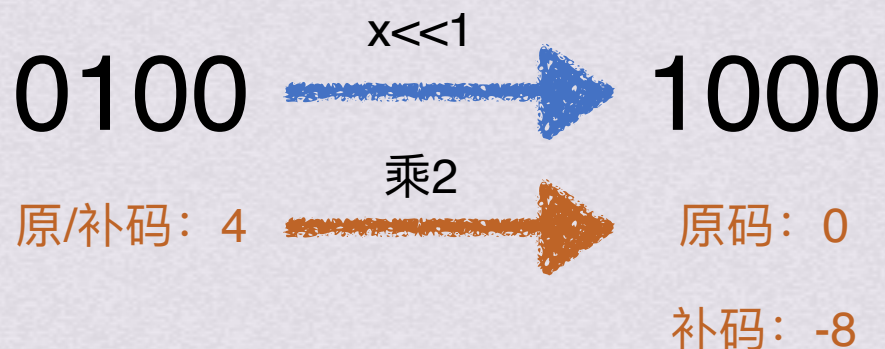
1. 定点数的移位操作

左移

$$x \ll k$$

x向左移动k位，丢弃最高的k位，并在右端补k个0

左移相当于乘2



若左移前后符号位不同，则发生溢出



定点数的运算

1.定点数的移位操作 1.2 右移

右移

$x \gg k$

x向右移动k位

逻辑右移 在左端补k个0 一般用于无符号数!

算数右移 在左端补k个最高位的值

1100 $\xrightarrow{\text{逻辑} x \gg 2}$ 0011

无符号数: 12 $\xrightarrow{\text{除以4}}$ 无符号数: 3

右移相当于除以2



定点数的运算

1.定点数的移位操作 1.2 右移

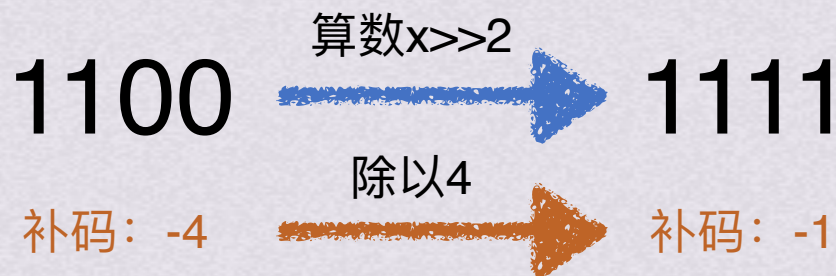
右移

$x \gg k$

x向右移动k位

逻辑右移 在左端补k个0

算数右移 在左端补k个最高位的值 一般用于补码!



右移相当于除以2



2.补码加减运算



定点数的运算

2.补码加减运算 2.1 补码加减运算法则

补码加减运算法则

$$[x]_{\text{补}} + [y]_{\text{补}} = [x+y]_{\text{补}}$$

$$[x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$

$[x]_{\text{补}}$ 转换成 $[-x]_{\text{补}}$: 全部位取反, 末位+1



定点数的运算

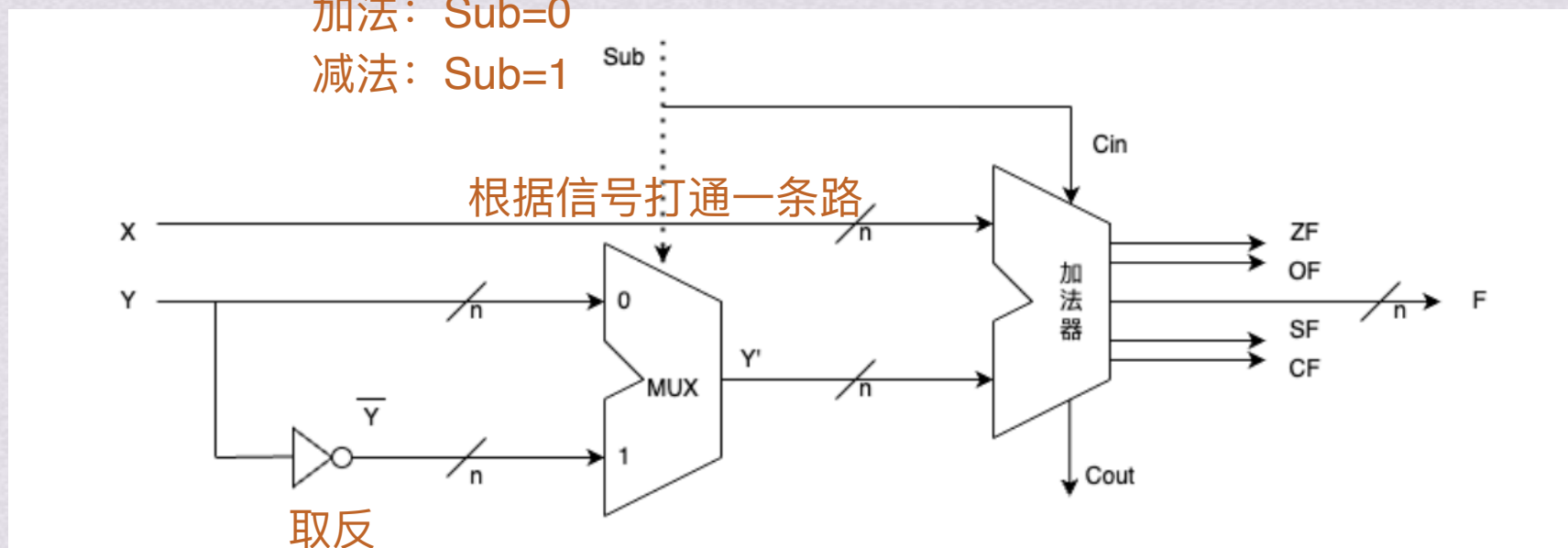
2.补码加减运算 2.2 补码加减运算电路

补码加减运算电路

该电路可以实现带符号数和无符号数的加减运算

加法: $\text{Sub}=0$

减法: $\text{Sub}=1$





定点数的运算

2.补码加减运算 2.3 溢出判别

溢出判别

仅有四种溢出情况

正数+正数=负数

负数+负数=正数

正数减负数=负数

负数减正数=正数



定点数的运算

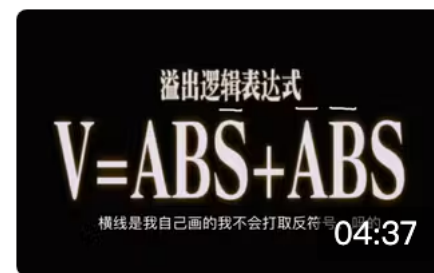
2.补码加减运算 2.3 溢出判别

溢出判别

$$V = A_s B_s \overline{S_s} + \overline{A_s} \overline{B_s} S_s$$

设A的符号为 A_s ，B的符号为 B_s ，运算结果的符号为 S_s

$V=1$ 说明溢出， $V=0$ 说明没有溢出



我猜你还没搞懂溢出逻辑表达式如何推导

4973

2023-5-15

推导过程补充



定点数的运算

2.补码加减运算 2.3 溢出判别

溢出判别

设置双符号位 $Ss1$ 和 $Ss2$ ，最高位的 $Ss1$ 代表真实的符号

若 $Ss1Ss2=00$ ：代表结果为正数，无溢出

若 $Ss1Ss2=01$ ：代表结果正溢出

若 $Ss1Ss2=10$ ：代表结果负溢出

若 $Ss1Ss2=11$ ：代表结果为负数，无溢出



定点数的运算

2.补码加减运算 2.3 溢出判别

溢出判别

采用一位符号位，根据数据位的进位情况判断溢出

若符号位的进位 C_s 与最高数位进位 C_1 相同，则没有溢出，否则溢出



3.乘法原理与电路

不建议深入，投入产出比太高！



定点数的运算

3.乘法原理与电路 3.1 原码乘法

$$X=-0.1101, Y=0.1011, \text{求} X*Y$$

$$X*Y=X*0.1011$$

$$=0.1*X+0.00*X+0.001*X+0.0001X$$

$$=0.1(X+0.0*X+0.01*X+0.001*X)$$

$$=0.1(X+0.1(0*X+0.1*X+0.01*X))$$

$$=0.1(X+0.1(0*X+0.1(X+0.1(X+0))))$$

乘法由【加法】+【右移】实现



定点数的运算

3.乘法原理与电路 3.1 原码乘法

中间每次计算后都会
直接加和，称为部分积

$$\begin{array}{r}
 \begin{array}{r}
 0.1\ 0\ 1\ 1 \\
 \times 0.1\ 1\ 0\ 1 \\
 \hline
 \end{array}
 \end{array}$$

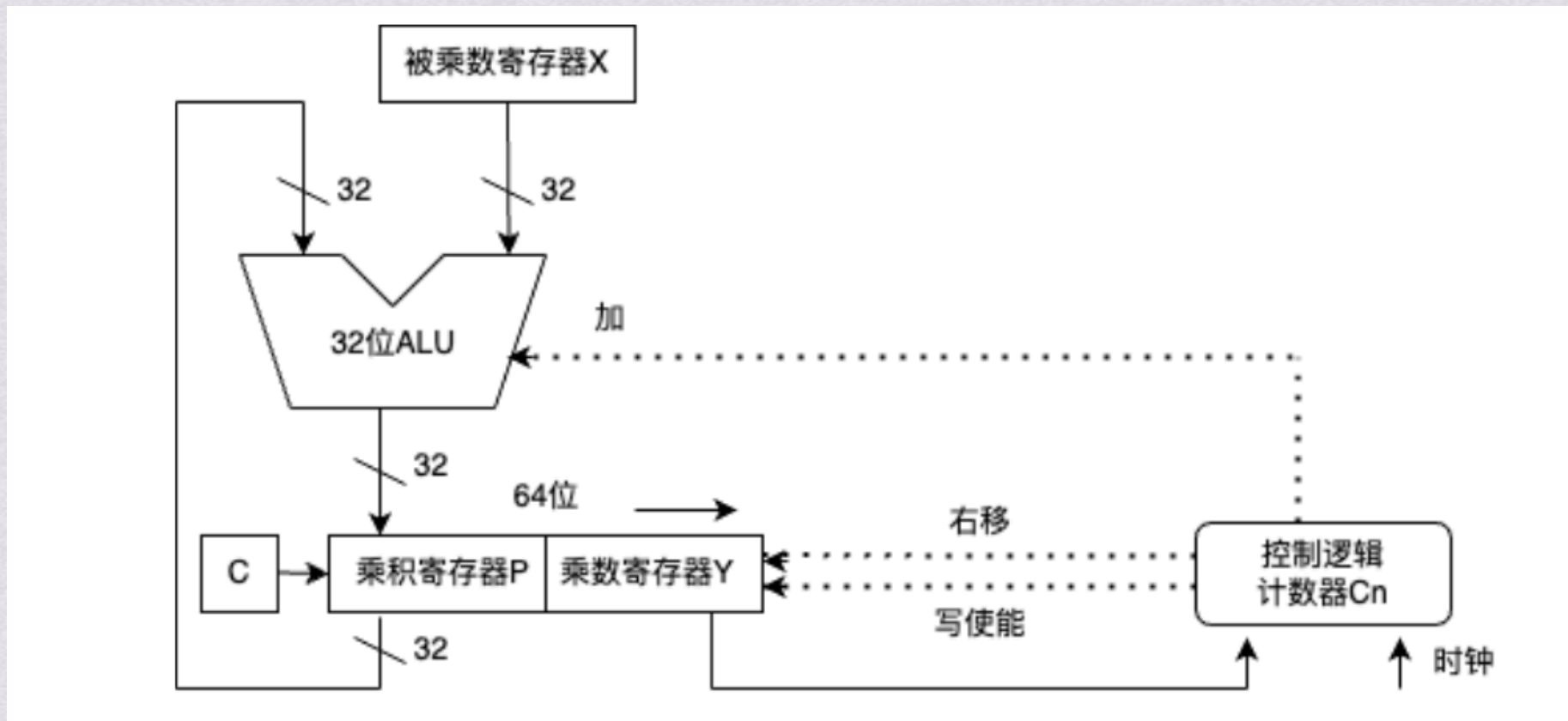
被乘数 $X=0.X_1\ X_2\ X_3\ X_4 = 0.1\ 0\ 1\ 1$
乘数 $Y=0.Y_1\ Y_2\ Y_3\ Y_4 = 0.1\ 1\ 0\ 1$

$$\begin{array}{r}
 1\ 0\ 1\ 1 \text{ --- } X*Y_4*2^{-4} \\
 0\ 0\ 0\ 0 \text{ --- } X*Y_3*2^{-3} \\
 1\ 0\ 1\ 1 \text{ --- } X*Y_2*2^{-2} \\
 1\ 0\ 1\ 1 \text{ --- } X*Y_1*2^{-1} \\
 \hline
 0.1\ 0\ 0\ 0\ 1\ 1\ 1\ 1
 \end{array}$$



定点数的运算

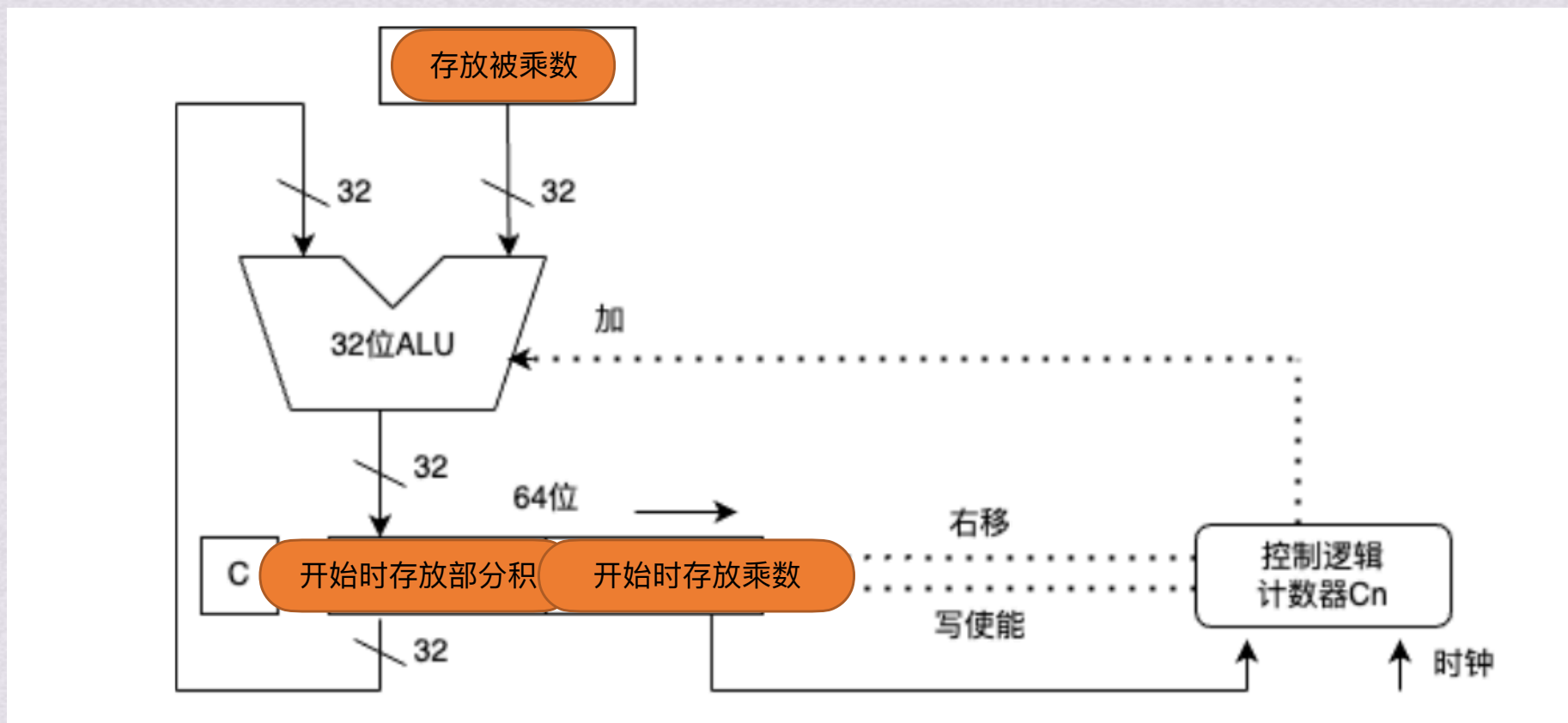
3.乘法原理与电路 3.1 原码乘法





定点数的运算

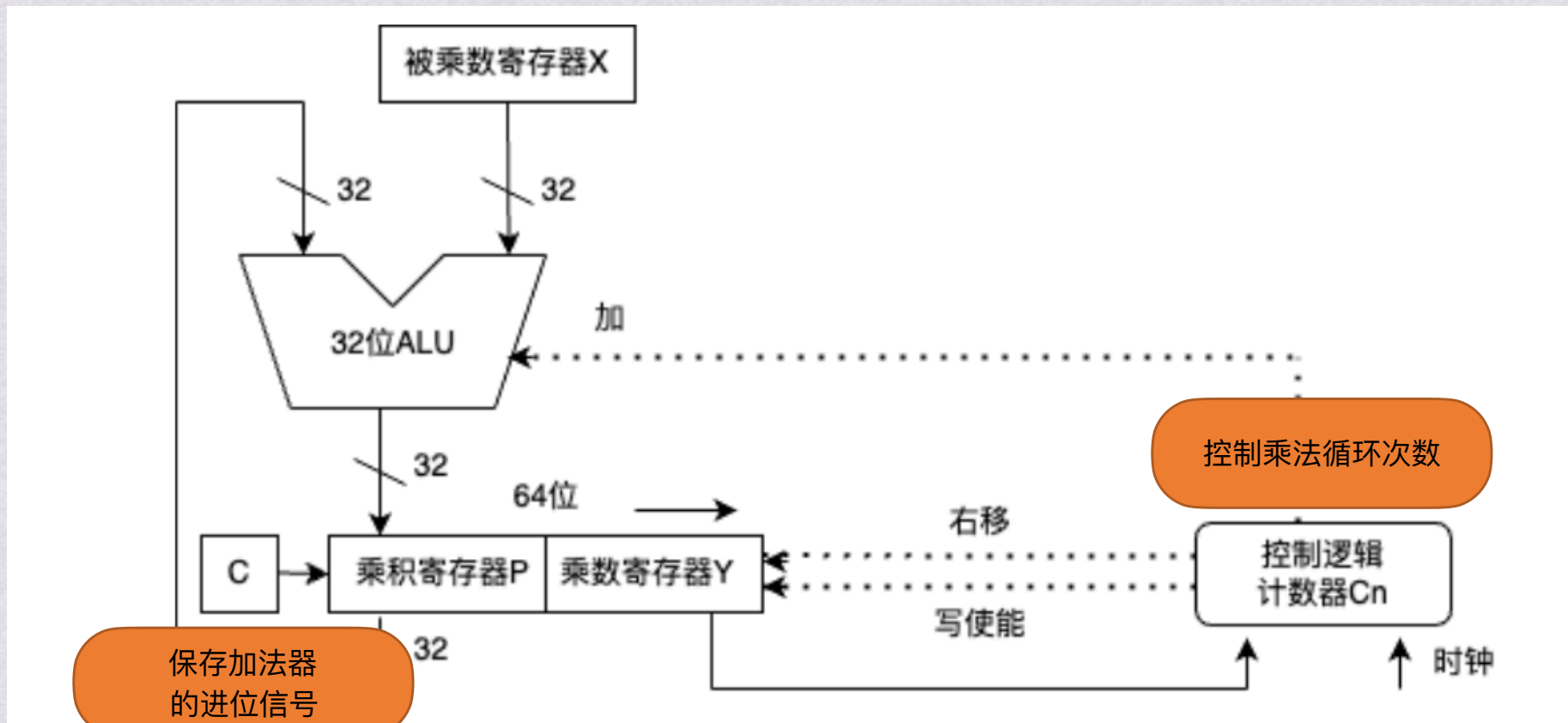
3.乘法原理与电路 3.1 原码乘法





定点数的运算

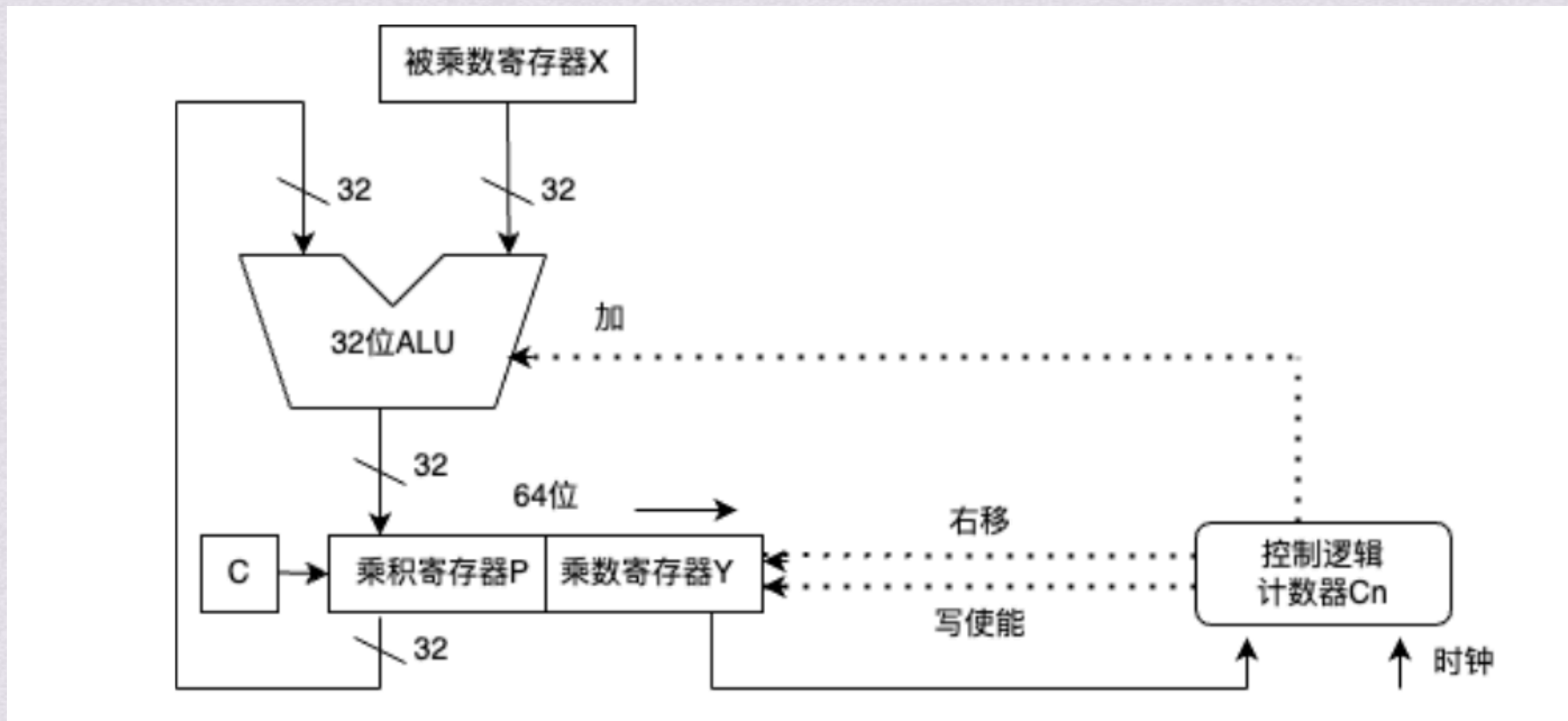
3.乘法原理与电路 3.1 原码乘法





定点数的运算

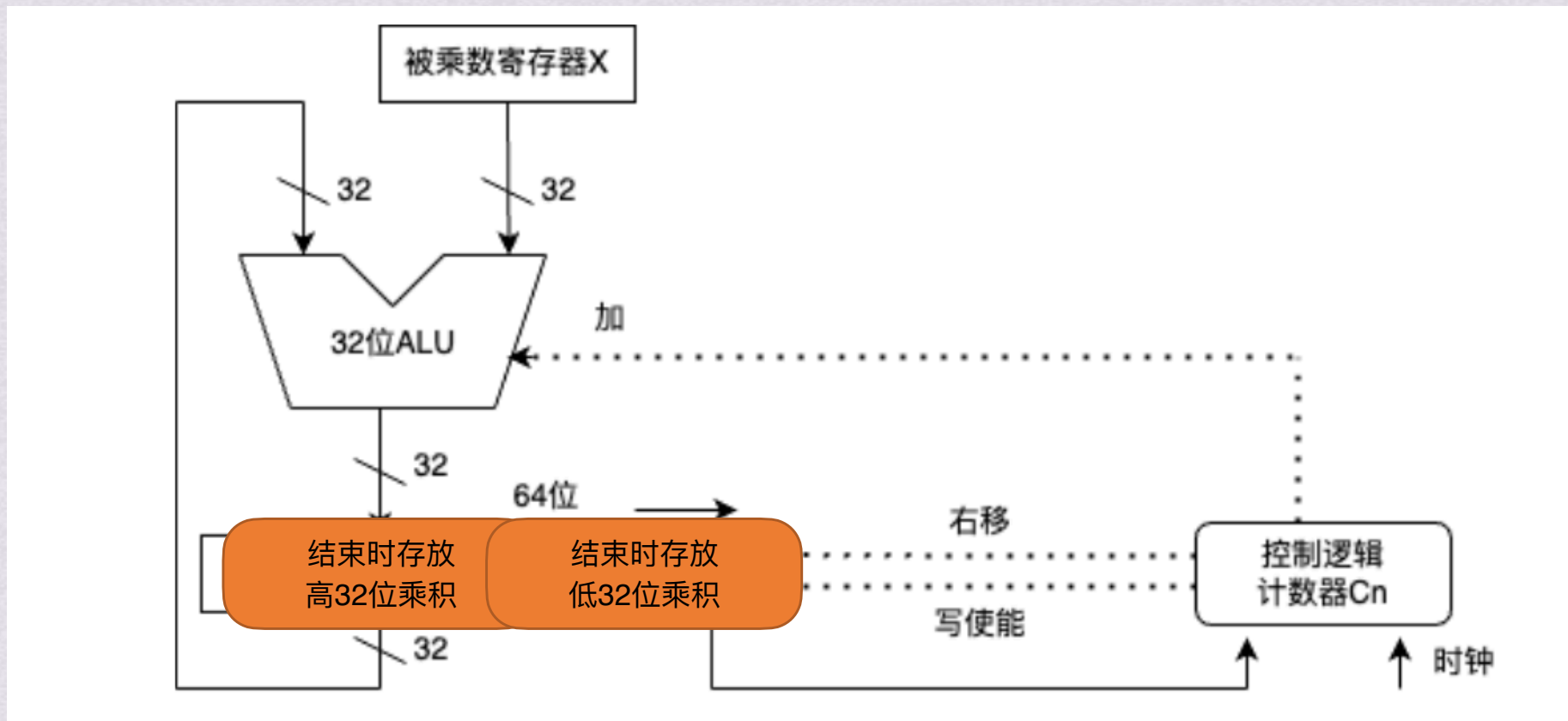
3.乘法原理与电路 3.1 原码乘法





定点数的运算

3.乘法原理与电路 3.1 原码乘法





定点数的运算

3.乘法原理与电路 3.1 原码乘法

Y最低位为1则加上被乘数，右移
Y最低位为0则加上0，右移

$X = -0.1101$, $Y = 0.1011$, 求 $X * Y$

每次【判断+加法】后
都要把部分积更新到P中再【右移】

0.0000



正在操作：判断

判断计数：0 加法计数：0 右移计数：0



定点数的运算

3.乘法原理与电路 3.1 原码乘法

Y最低位为1则加上被乘数，右移
Y最低位为0则加上0，右移

$X = -0.1101$, $Y = 0.1011$, 求 $X * Y$

每次【判断+加法】后
都要把部分积更新到P中再【右移】



```

      0 . 0100
    + 0 . 1101
    -----
      0 . 1000      1111
    
```

结果的绝对值部分

【判断+加法+右移】进行n轮
标志着算法结束

正在操作：右移

判断计数：4 加法计数：4 右移计数：4



定点数的运算

3.乘法原理与电路 3.1 原码乘法

总结

机器字长为 n ，共进行【判断】+【加法】+【右移】 $n-1$ 轮

判断 { 最低位为1：加被乘数，右移
最低位为0：加0，右移

根据乘数寄存器Y的最低位判断每一步的操作

更新 每次【判断】+【加法】执行后，先将部分积更新到乘积寄存器P，再整体进行右移

符号 结果的符号位等于乘数与被乘数的符号异或

定点数的运算

3.乘法原理与电路3.2 补码乘法

溢出判别

设 $[x]_{补}=Xs.X_1X_2...X_n$ ， $[y]_{补}=Ys.Y_1Y_2...Y_n$ ，运算规则如下：

- 1.符号位参与运算
- 2.被乘数取双符号位，乘数取单符号位
- 3.乘数末位增设附加位，初值为0

• 次低位	• 最低位	• 操作
• 0	• 0	• 部分积加0，部分积右移一位
• 0	• 1	• 部分积加 $[x]_{补}$ ，右移一位
• 1	• 0	• 部分积加 $[-x]_{补}$ ，右移一位
• 1	• 1	• 部分积加0，部分积右移一位

机器字长为n，重复n轮后得出结果，注意第n轮不进行右移



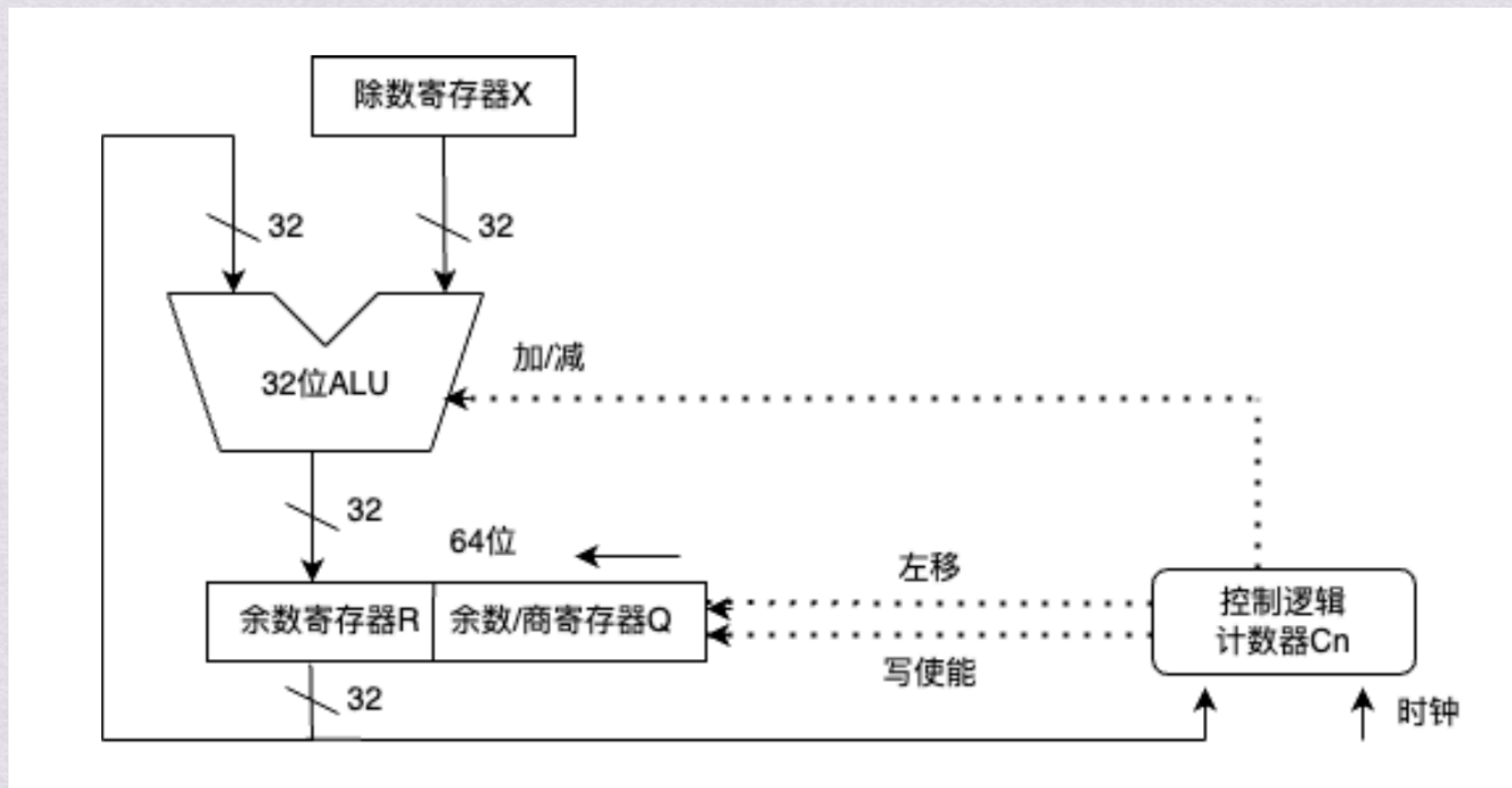
4.除法原理与电路

不建议深入，投入产出比太高！



定点数的运算

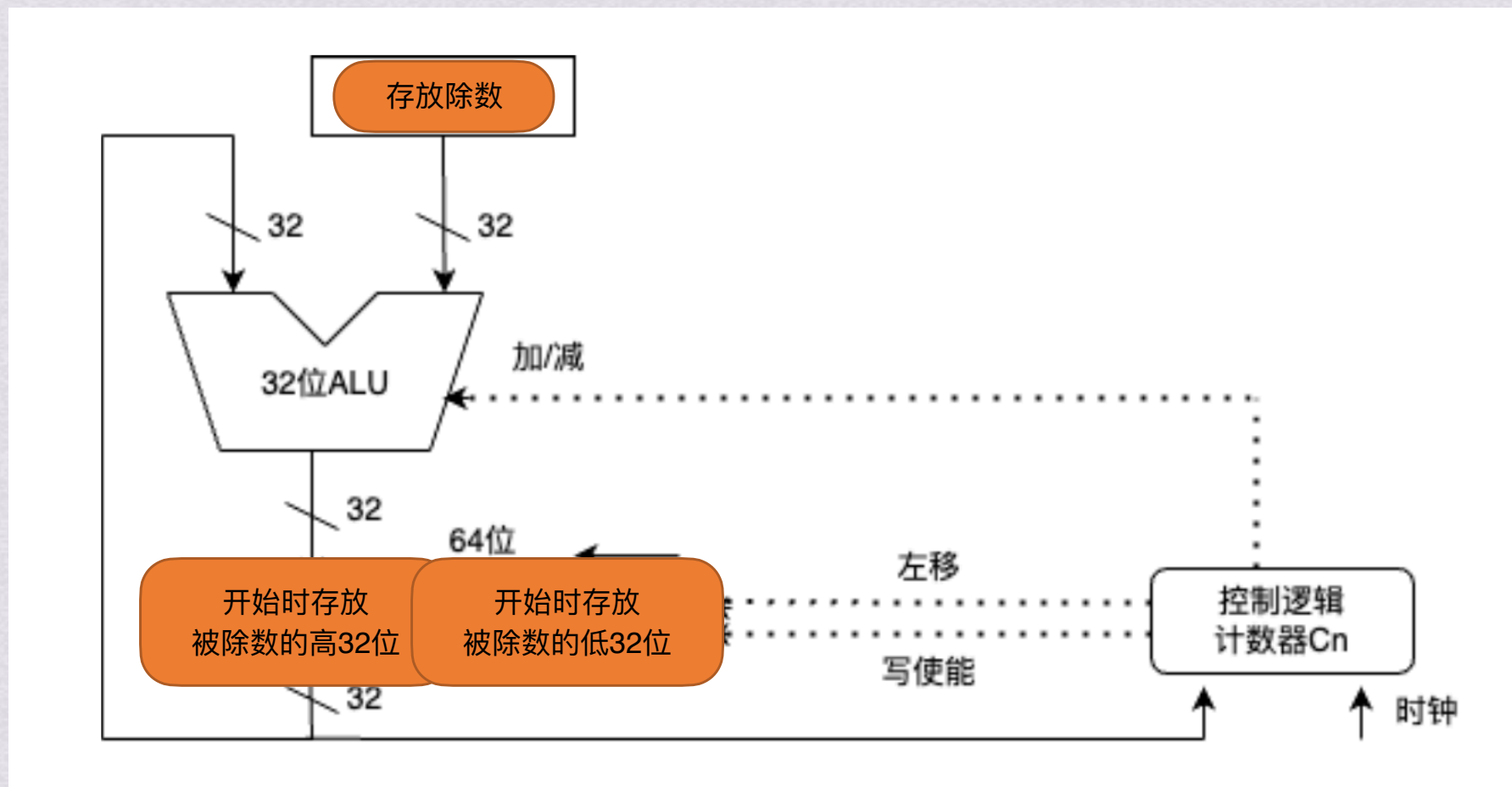
4.除法原理与电路





定点数的运算

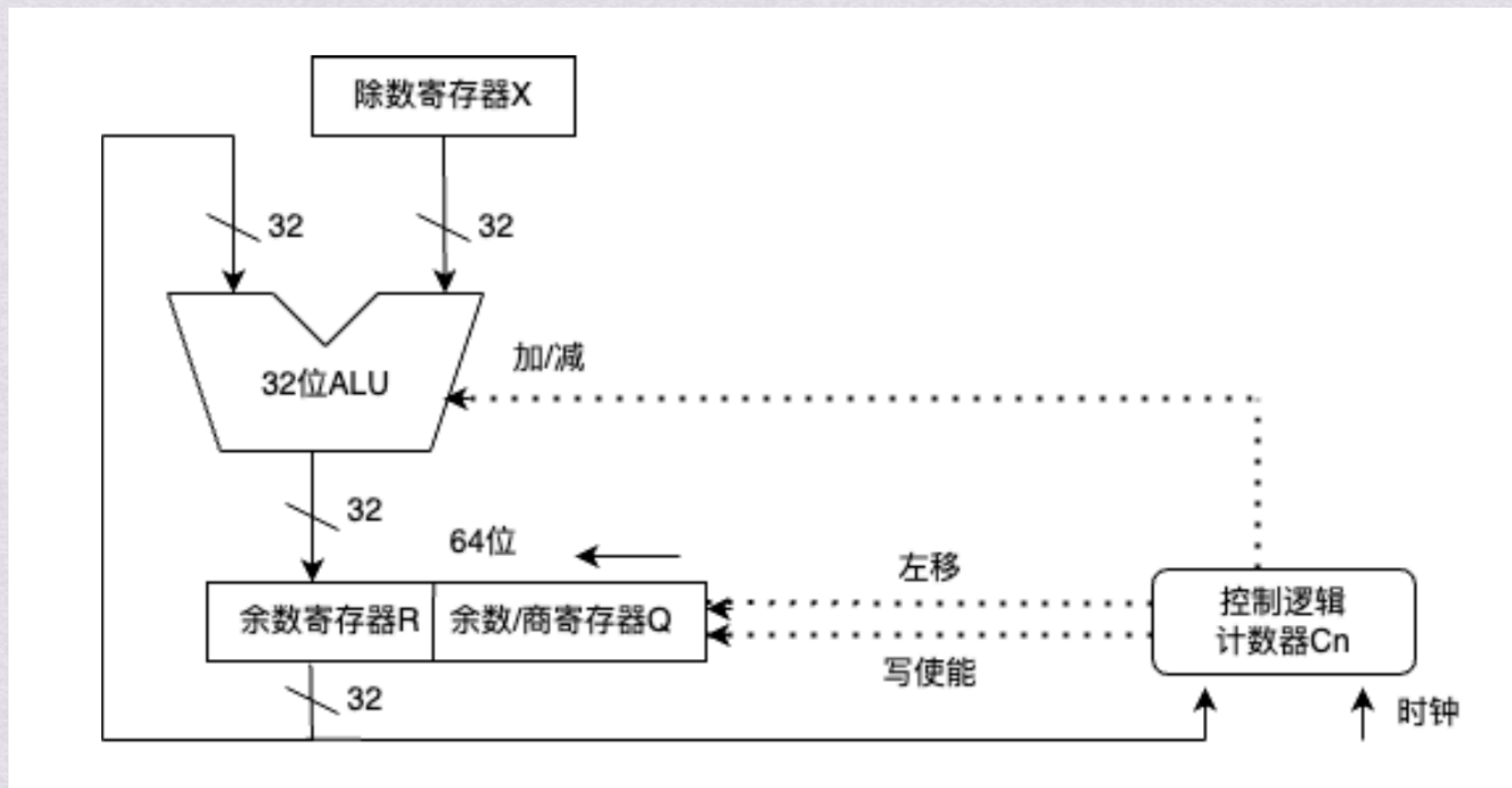
4.除法原理与电路





定点数的运算

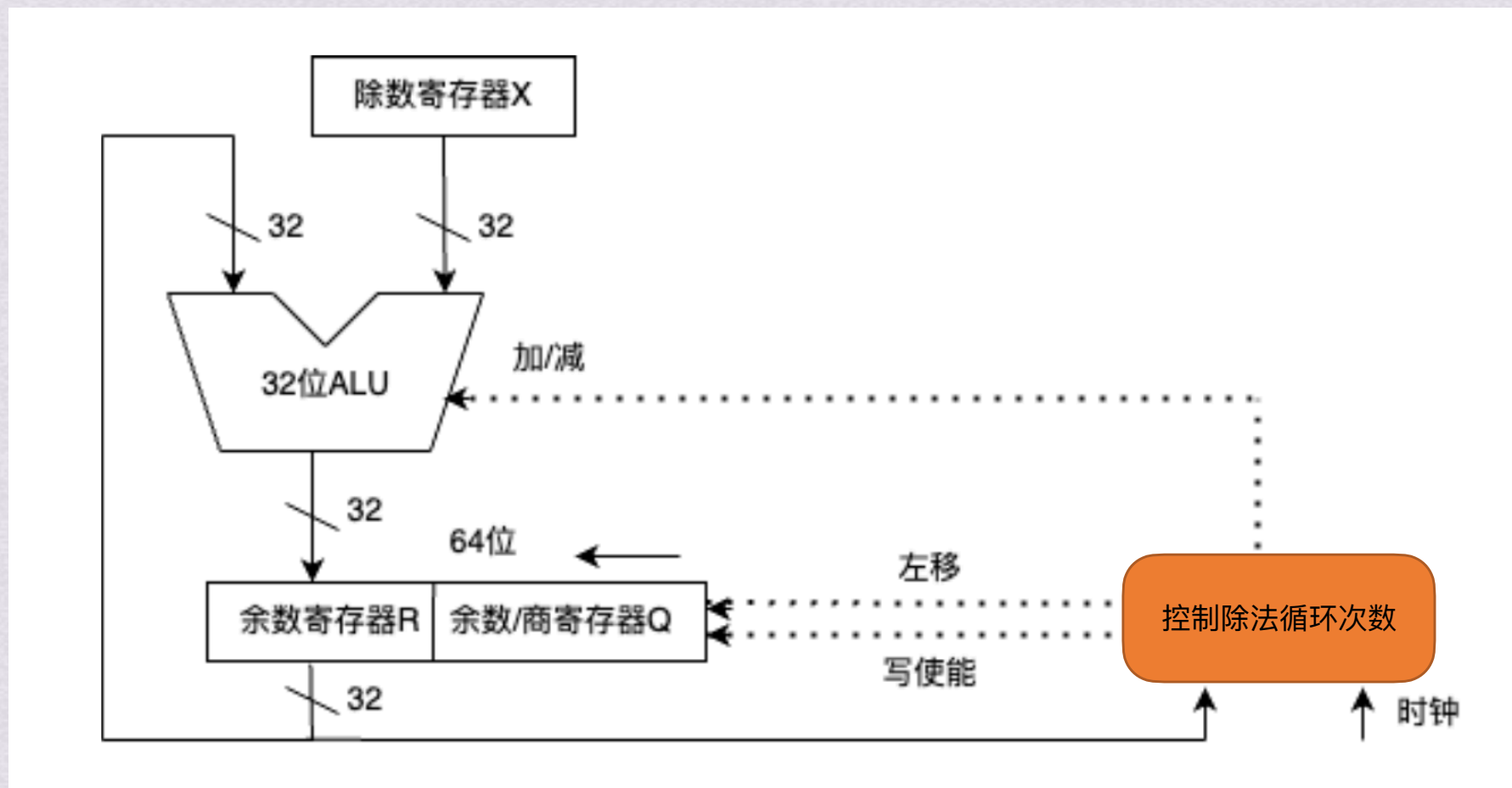
4.除法原理与电路

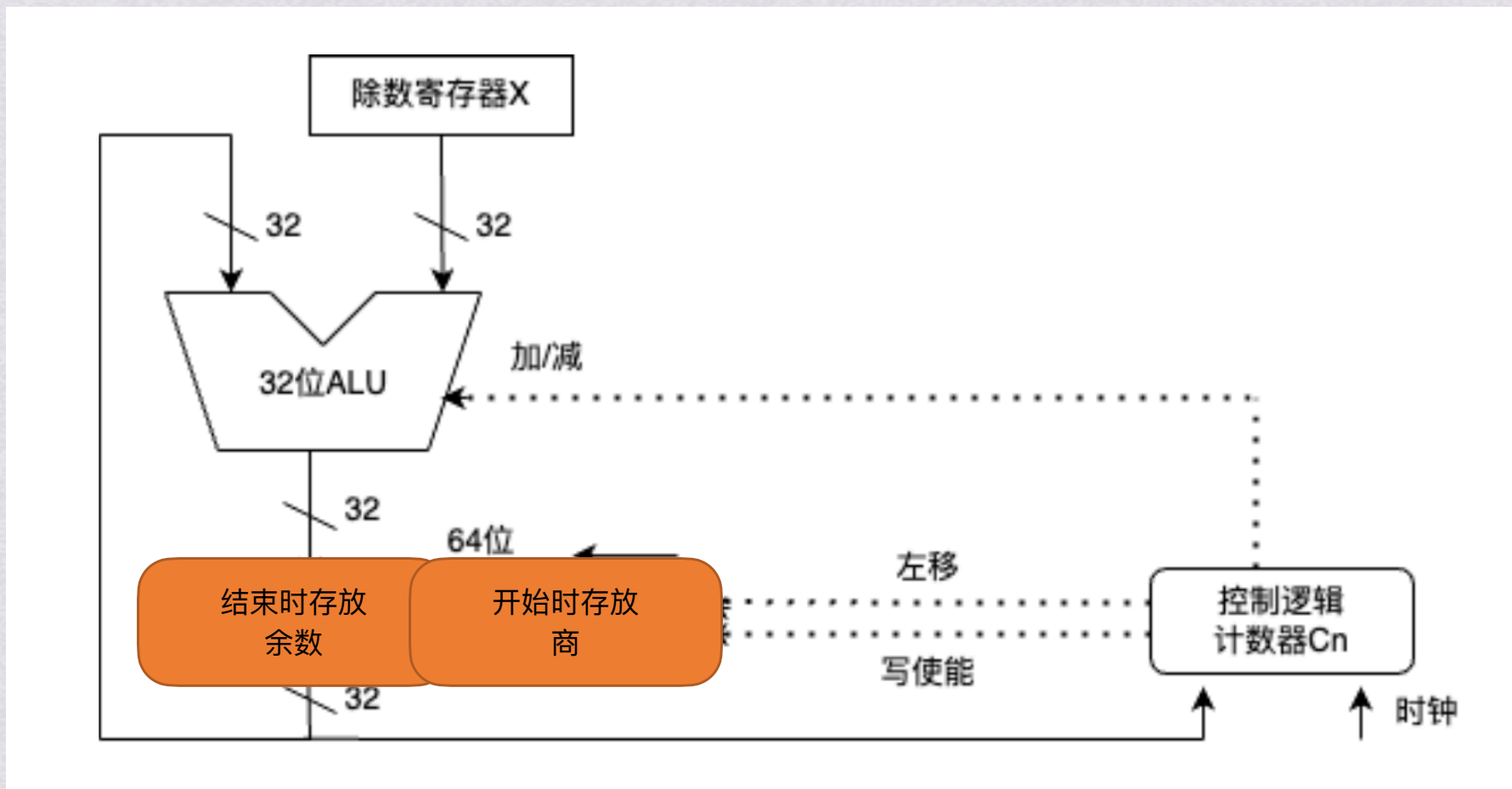
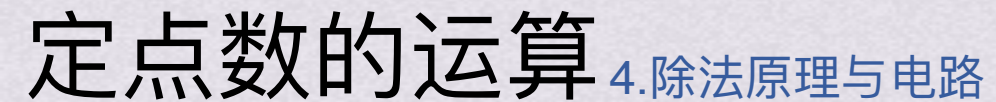




定点数的运算

4.除法原理与电路







定点数的运算

4.除法原理与电路

原码除法运算-不恢复余数法

- 设被除数 $[x]_{\text{原}} = X_s.X_1X_2\dots X_n$, $[y]_{\text{原}} = Y_s.Y_1Y_2\dots Y_n$, 运算规则如下:
 - 商的符号由两个数的符号位异或而成。
 - 先用被除数减去除数, 当余数为正时, 商上1, 余数和商左移一位, 再减去除数; 当余数为负时, 商上0, 余数和商左移一位, 再加上除数。
 - 当第 $n+1$ 步 (机器字长为 $n+1$) 余数为负时, 需加上 $|y|$ 得到正确的余数 (正确的余数和被除数同号)。



定点数的运算

4.除法原理与电路

补码除法运算-加减交替法

- 设被除数 $[x]_{\text{原}} = X_s.X_1X_2\ldots X_n$, $[y]_{\text{原}} = Y_s.Y_1Y_2\ldots Y_n$, 运算规则如下:
 - 符号位参与运算, 除数与被除数均用补码表示, 商和余数也用补码表示。
 - 被除数和余数采用双符号位, 商采用单符号位。
 - 若被除数和除数同号, 则被除数减去除数; 若被除数与除数异号, 则被除数加上除数。
 - 若余数与除数同号, 则商上1, 余数和商左移一位, 余数减去除数; 若余数与除数异号, 则商上0, 余数和商左移一位, 余数加上除数。
 - 重复执行上一步操作 n 次 (机器字长为 $n+1$) 。
 - 一般情况下, 商的末位恒置1。



定点数的运算

关于乘法和除法



定点数的除法运算（恢复余数法，加减交替法）

📺 1.2万

2022-8-24



定点数的乘法运算

📺 1.5万

2022-8-19